Pragnosia: Making a Diagonal-Complex Spin Recurrence the Load-Bearing Core of a Small Language Model

Ashish

Independent Research

Preprint · 2026 · correspondence: ashish99anonymous@gmail.com

Abstract. We study a transformer/SSM hybrid in which a rotational "spin" recurrence a diagonal-complex linear-recurrent unit run as a parallel associative scan is the *core token-mixer*, with attention only a periodic helper. The work has a sharp, honest arc. We first scaled the hybrid *the wrong way*: a single spin carrier placed after a transformer stack, whose parameter share collapses `5.76%` → `1.90%` → `0.37%` from 21M to 1.4B; at 1.4B a causal probe shows the carrier contributes only **0.028% of the loss** vestigial as built. A param-matched ablation then shows the *intended* design, spin as the mixer (`carrier="spin_dominant"`), reaches val perplexity **219** vs **279** for an attention-only baseline at matched parameters spin earns its weights when it is the core a *quality* win against a matched transformer. A control that swaps the complex carrier for a *real, no-phase* recurrence in the same placement also beats attention (3.5×, matching the complex carrier at this short budget), so *at the tested budget* the effect is about *placement*, not the complex phase specifically. Separately and this is *causal attribution, not a quality win* at scale: a **284M** spin-dominant model (val ppl 24.54 on a 176.6B-token corpus) in which *ablating the spin carrier sends perplexity 25 → 7716 (306×)* while ablating attention costs only **3.4×** the carrier is the load-bearing core, the exact inverse of the 1.4B drift. We keep the two claims distinct: the 284M result shows the carrier is *used*, not that the design *wins* a matched `carrier="none"` transformer at 284M, which would show the quality win persists at scale, is the key experiment still to run. On this substrate runs a *proposed* self-calibrating controller (mechanisms below; their validation is preliminary): continual learning by surprise-modulated plasticity with generative replay, a persistent in-weights episodic memory, an introspection layer (a confidence self-estimate, deliberation, and idle self-generation), and function-preserving self-growth that fires only on probe-confirmed saturation. Every internal threshold is derived from data and re-derived as the model grows; nothing is hand-set. We report results and limits plainly: the spin-dominant win is small-scale, short-budget, single-seed and must be replicated at 1B; the grown models are undertrained for their size (multi-hop reasoning is weak); the model's honesty signal, while functional, does not yet separate knowledge from confident confabulation at 284M; and the long-context advantage is *structural* the implemented cross-window carry is not yet trained and currently *hurts* (next-window ppl 62.6 with it vs 58.0 without).

1. Introduction

The dominant paradigm in AI is the large language model: a transformer [1] trained to predict the next token. LLMs are capable but, relative to the goal of an adaptive, persistent learner, three properties are missing by construction. They are *frozen* once pretraining ends they cannot acquire a fact without fine-tuning or prompt-stuffing, and naive updates cause catastrophic forgetting [2]. They are *over-confident* lacking a model of their own uncertainty, they fabricate rather than abstain. And their computation is opaque there is no standard intervention separating in-state computation from memorized retrieval.

Pragnosia is an attempt to build the missing properties into the substrate itself, even at small scale. Its contributions are: (i) a diagonal-complex rotational recurrence used as the *core token-mixer* via a parallel scan (§2); (ii) a measured analysis of *where that carrier earns its parameters* our first scaled model drifted to attention-dominant with a causally-negligible carrier, while a param-matched ablation shows the intended *spin-dominant*

design wins by ~21%, now confirmed load-bearing at 284M by ablation, and shown to generalize to a different (real, no-phase) recurrence in the same placement (§3); (iii) a self-calibrating controller with surprise-modulated continual learning, a persistent in-weights episodic memory, an introspection layer, and function-preserving self-growth, with no hand-set constants (§4); and (iv) an honest account of what works and what does not (§5–7). We release the system and every measured number.

Table 1. Summary of the central results. These are *two* claims, kept distinct in the text: a small-scale *quality* win (spin-dominant beats a matched transformer, rows 3) and a large-scale *causal-dominance* property (the carrier is load-bearing, rows 4–5). The 284M rows show the carrier is *used*, not that the design wins at scale a matched `carrier="none"` transformer at 284M is the experiment that would join the two.

Result	Value	What it shows
Carrier parameter share, 21M→1.4B	5.76% → 0.37%	a fixed single carrier dilutes as the model scales
Carrier causal signal at 1.4B	0.028% of loss	vestigial as built the wrong placement
Param-matched ablation (~37M)	219 vs 279 ppl	spin-dominant by ~21% small, controlled
284M spin-dominant	val ppl 24.54	the intended build, trained at real scale
284M carrier ablation	306× vs 3.4×	the carrier is load-bearing the right placement
1.4B "wrong build"	val ppl 17.35	bigger, yet the carrier is vestigial

2. Related Work

Recurrent and state-space sequence models. The carrier is a diagonal linear recurrence in the lineage of structured state-space models S4 [8] and its diagonal/simplified variants, the Linear Recurrent Unit (LRU) [4], and S5 and of recent recurrence-as-mixer models: Mamba [3] (selective SSM), RWKV [10], RetNet [11], and the linear-attention / DeltaNet and xLSTM families; Hyena reaches the same goal with long implicit convolutions. Our carrier itself is *not* novel it is a standard diagonal-complex LRU run as a parallel associative scan.

Where we differ placement and verification. These models are typically the *sole* mixer, or interleaved uniformly with attention. Our question is narrower and, we believe, under-studied: in a hybrid, is the recurrence carrying the computation, or is it a vestigial side-channel? We contribute (i) a measured account that a single carrier's contribution *collapses with scale* when placed after a transformer stack, and (ii) a causal protocol carried-state intervention on single-carrier models, direct ablation on the spin-dominant design to test whether the mixer is load-bearing. Our headline (306× ablation cost vs 3.4× for attention at 284M) is a property of *placement*, not of the recurrence.

Continual learning and memory. The controller's no-forgetting mechanism is generative replay / pseudo-rehearsal [6]; its surprise-gated learning rate is in the spirit of neuromodulation [7]; its in-weights episodic store is a fast-weight associative memory (related to modern Hopfield networks [12]), not retrieval-augmented generation. We do not claim these mechanisms are individually novel the contribution is their integration into one self-calibrating model with no hand-set constants.

3. The Spin Carrier

A diagonal-complex recurrence, run as a parallel scan.

The carrier is a linear-recurrent unit [8] over a complex hidden state. For input `x` (layer-normed to `y`), with an input-dependent gate `g=σ(G y)`, the per-channel recurrence and read-out are

$$b_t = g \odot (U_{re}\, y_t + i \cdot U_{im}\, y_t), \quad h_t = \lambda \odot h_{t-1} + b_t,$$

$$\lambda_j = \exp(-\exp(\nu_j)) \cdot e^{i\theta_j}, \quad \text{out}_t = C\cdot[\text{Re } h_t \,;\, \text{Im } h_t], \quad x_t \leftarrow x_t + \sigma(\gamma)\cdot\text{out}_t. \tag{2}$$

Each channel `j` is thus a *damped complex oscillator* at its own frequency θ_j and decay $|\lambda_j| \in (0,1)$: information is carried in the evolving *phase* of the state, read back to the real stream through `C` and added by a learned gate $\sigma(\gamma)$. Because λ is diagonal the recurrence is matmul-free $O(T\cdot d)$, not $O(T\cdot d^2)$ *and* associative, so it is computed not by a `T`-step loop but by a log-depth **parallel associative scan** (a Hillis–Steele scan over the sequence). An in-place variant is exact and ~20% faster on the forward pass but breaks autograd; we keep the materializing scan. This is the only recurrence in the system; an earlier dense, tanh-nonlinear sequential variant is retained in version history but is not used.

Why phase.

Encoding state in the phase of a damped oscillator, rather than a settled activation, has three motivations. (i) *Capacity without interference*: a bank of oscillators at distinct frequencies represents many independent slowly-varying quantities side by side, much as Fourier components are orthogonal. (ii) *Stable, multi-timescale memory*: with $|\lambda|<1$ the state decays gracefully rather than exploding or collapsing, and the per-channel decay ν is learned, so different channels hold information over different horizons. (iii) *Parallelizable dynamics*: a diagonal-complex recurrence is associative, which is exactly what admits the log-depth parallel scan that makes training practical a property a nonlinear recurrence lacks. We do not claim phase is *necessary*; we note it is a convenient, trainable basis with these properties, and the empirical question of this paper is whether such a mixer, placed as the core, carries the computation.

Architecture and modes.

A `SpinBlock` uses the carrier as its token mixer (a strong gate, $\sigma(\gamma)\approx 0.88$) followed by an MLP; a standard attention `Block` is the alternative. The model exposes one config flag, `carrier ∈ {none, single, per_block, spin_dominant}`, wired `pragnosia.json → train_pragnosia.py → grow.py`. `none` is a transformer; `single` places one carrier after the stack; `spin_dominant` the intended design makes the carrier the mixer in 3 of every 4 layers, with attention every 4th as a helper. At long context the carrier is $O(T\cdot d)$ versus attention's $O(T^2\cdot d)$, and at inference it is recurrent: $O(1)$ per token, constant memory, no KV-cache growth. `torch.compile` fuses the scan for ~2× training throughput measured `49K → ~100K tok/s` (bs 24) on a consumer RTX 4060. It is *not* faster than attention on short (256-token) training sequences; its intended wins are long context and recurrent inference (the long-context win is *structural* here realizing it in a trained model also requires training with the cross-window carry, which we have not yet done; see §Limitations).

4. Where the Spin Carrier Earns Its Parameters

The scaled model drifted to attention-dominant.

Our first scalable design placed a *single* gated spin carrier after a transformer stack. A single carrier is one fixed `d`-shaped layer (~5.25M params), so as the model grows by depth and width its *parameter share collapses*: 5.76% (21M) → 1.90% (276M) → 0.37% (1.4B). We measured its causal contribution by a *carried-state intervention* on the `single`-carrier model (transplant the carrier state across a segment boundary; a causal carrier shifts the continuation): at 1.4B the ordering own < swapped < random still holds *directionally*, but the contribution is only `0.00086` NLL **0.028% of the loss**, down ~20× from 276M. As built, the carrier is *causally negligible at scale*: the model is a transformer with a vestigial side-channel. (We thank an external reviewer for flagging this.) The same 1.4B run also over-grew: an earlier trigger grew on any validation plateau rather than true saturation, so an undertrained model ballooned to 1.4B (48 layers, mlp_mult 12) long before the ≈28B tokens needed to fill it over-growth, not a result. §4 gives the corrected growth rule.

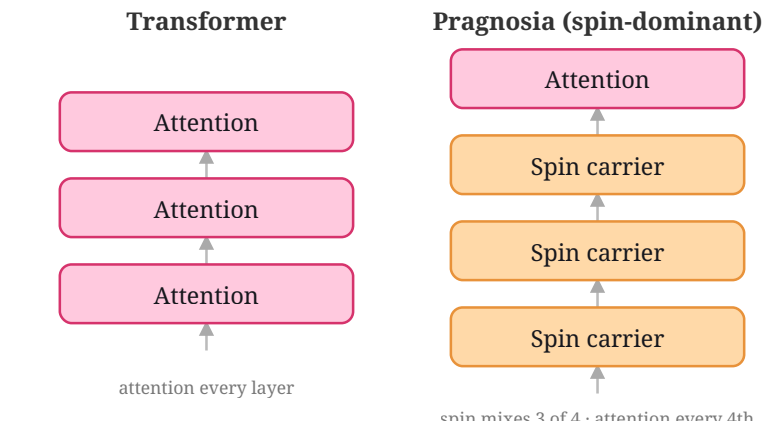

Figure 1. The architectural difference. A Transformer mixes tokens with attention in every layer; Pragnosia's spin-dominant design makes the diagonal-complex spin carrier the token-mixer in 3 of every 4 layers, with attention a periodic helper. The ablation (Fig. 1) shows the carrier not attention is load-bearing in this arrangement.

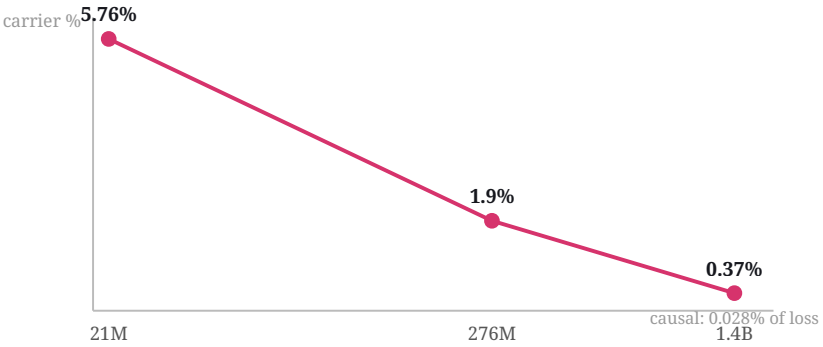

Figure 2. A fixed single spin carrier is one d-shaped layer, so as depth and width grow its parameter share collapses (5.76% → 1.90% → 0.37% from 21M to 1.4B) and its measured causal contribution falls to 0.028% of the loss at 1.4B the motivation for making the carrier the core rather than a side-channel.

The intended design is spin-dominant and wins at matched parameters.

The drift inverts the intended architecture, where spin is the core mixer. We test the intended design with a clean param-matched ablation (same corpus, same budget, `d=512`, single seed):

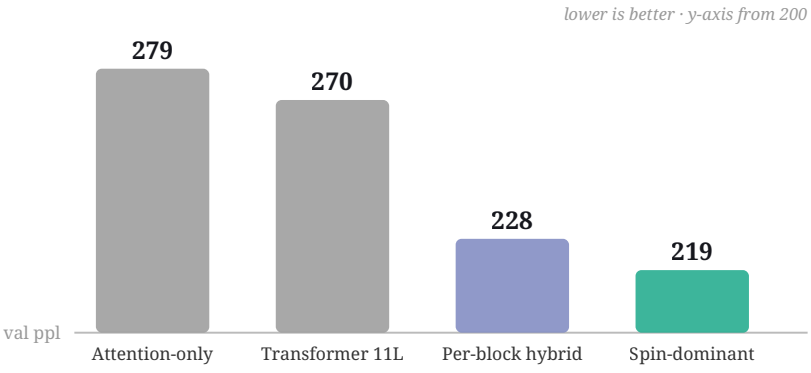

Figure 3. Param-matched ablation (~37–46M, same corpus and budget, single seed). Spin as the token-mixer (219) beats the attention-only baseline (279) by ~21%, and the larger per-block hybrid (228) and param-matched transformer (270), with fewer or equal weights.

Spin-dominant wins by ~21% over the attention-only baseline at matched parameters and beats the larger per-block hybrid and param-matched transformer with fewer weights. **The carrier earns its parameters when it is the core mixer, not a side-channel.** This ablation is small-scale and short-budget though the core comparison is now confirmed seed-robust (next) and must be replicated at 200M–1B before it is proven; we report it as a strong, properly-controlled signal and the corrective to the drift.

Seed robustness.

The ablation above is single-seed. To check it is not an artifact of one initialization or of parameter count we re-ran the comparison across **3 seeds** for four architectures: attention-only, a parameter-matched 11-layer transformer, a per-block side-channel carrier, and spin-dominant. Each is at a matched short budget (~30M tokens) with a per-architecture learning rate set by a Smith range-test (the recipe behind the numbers above). The ordering is highly reproducible:

Table 2. Param-matched ablation. Spin as the token mixer is the most parameter-efficient.

Design	Params	Val ppl
Attention-only (transformer)	35.7M	279.0
Spin-dominant (intended)	37.3M	**219.1**
Per-block hybrid (attn + spin each block)	46.2M	228.0
Param-matched transformer-only (11L)	45M	270

Table 3. Multi-seed robustness across the attention family vs spin-as-core (3 seeds each; per-architecture range-test lr; matched ~30M-token budget). Even a larger transformer and a side-channel carrier stay at 329–361; spin as the core wins ~3×.

Architecture	Params	lr	Val ppl (mean ± std)
Attention-only (8L)	35.7M	`1.0e-4`	360.9 ± 4.5
Transformer (11L, param-matched)	45.2M	`1.0e-4`	352.5 ± 2.4
Per-block carrier (side-channel)	46.2M	`1.0e-4`	329.3 ± 3.6
Spin-dominant (spin as core)	37.3M	`6.7e-3`	**115.7 ± 1.8**

Two readings. (i) **The gap is not seed noise, nor capacity** the standard deviation is ~1–2% of the mean, so the ordering barely moves across initializations; and crucially neither a *larger* transformer (11L, 45M) nor a side-channel carrier (per-block, 46M) escapes the attention-family band (329–361) despite having *more* parameters than spin-dominant (37M). The win is the placement, not the parameter count. (ii) **The gap is budget-dependent**: it is far larger here (~3.1×) than the ~21% above because this budget is much shorter the recurrence drives perplexity down with far fewer tokens, and attention narrows the gap as both train longer. We therefore treat this as a *robustness check* the win is consistent across seeds not a re-statement of the parameter-efficiency margin, whose magnitude depends on the token budget. We also note the two architectures prefer very different learning rates: the range-test gives the spin carrier `6.7e-3` versus the transformer's `1.0e-4`, a 67× difference consistent with state-space models tolerating higher rates part of spin-dominant's early-training speed is that it takes larger stable steps.

Generalization: at the tested budget, placement not phase.

A natural objection is that "placement matters" might be specific to this diagonal-*complex* carrier rather than a property of recurrent mixers in general. We test it directly: we replace the spin carrier with a diagonal-**real** recurrence ($h_t = \lambda \odot h_{t-1} + b_t$, real $\lambda \in (0,1)$, no phase) — a different recurrence family — placed identically as the core (3 of every 4 layers), under the same multi-seed protocol.

Table 4. Generalization control: a real, no-phase recurrence in the same core placement (3 seeds, per-architecture range-test lr, matched ~30M-token budget).

Core token-mixer	Params	Val ppl (mean ± std)	vs attention
Attention (transformer)	35.7M	360.9 ± 4.5	—
Complex spin carrier	37.3M	**115.7 ± 1.8**	**3.1×**
Real diagonal carrier (no phase)	34.1M	**102.2 ± 2.1**	**3.5×**

The real, no-phase recurrence as the core beats the attention baseline by **3.5×** — as much as the complex carrier (3.1×), and with *fewer* parameters (34.1M vs 37.3M). **At the tested budget, the placement effect generalizes to a different recurrence family** it is not an artifact of the complex/phase representation. We state this carefully: because the real recurrence here *outperforms* the complex one at this short budget, we claim only that *placement* (a recurrence as the core mixer) is what carries the computation at the tested scale and budget. Whether the complex phase provides advantages at larger scale, longer training, or longer contexts is an open question this experiment does not settle it shows the placement result survives swapping the recurrence, not that phase is useless. The phase remains a convenient, parallelizable basis (§2).

At scale: the spin carrier is the load-bearing core (284M).

We trained the intended design at real scale for the first time. A **284M** model (`d=768`, 20 layers, `mlp_mult=9`, `carrier="spin_dominant"` the carrier as mixer in 15 of 20 blocks, attention every 4th) trained to step 418,000 on a 176.6B-token chain-of-thought corpus, reaching **best validation perplexity 24.54** (about 24 tokens/param we report this as a description of the run, *not* as a Chinchilla-optimality claim: the corpus is deliberately reasoning-heavy and atypical, so the tokens/param ratio is not comparable to standard-corpus training). In the spin-dominant configuration the carrier is *intra-block* with no cross-segment state to transplant, so the carried-state intervention above does not apply; the equivalent causal measure is direct *ablation*.

The carrier carries essentially the whole computation. Gating the 15 SpinBlocks' carrier to ≈0 sends validation perplexity from **25 to 7716 306× worse**; zeroing the 5 attention blocks' output sends it from **25 to 85 3.4× worse**. At 284M the spin carrier is the *load-bearing core* and attention a minor helper the exact inverse of the 1.4B instance, whose attention-dominant-built carrier was vestigial (0.028% of the loss). The 284M is 5× smaller than the 1.4B yet is the correct build with a provably load-bearing carrier.

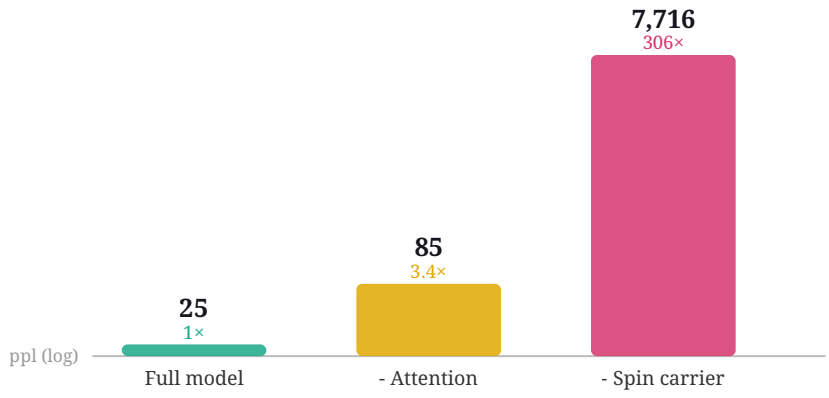

Figure 4. Causal ablation on the 284M spin-dominant model (log scale). Zeroing the spin carrier sends validation perplexity from 25 to 7,716 (306×); zeroing attention only to 85 (3.4×). The carrier is the load-bearing core.

5. A Self-Calibrating Adaptive Controller (Architecture and Mechanisms)

Status. This section describes an *architecture and its mechanisms*, not a validated set of results. Where we can measure a mechanism it works (self-growth fires only on probe-confirmed saturation; episodic writes/reads function); where the flagship behavior does not yet hold at this scale we say so plainly and mark it preliminary the honesty gate does not separate knowledge from confabulation at 284M (§6), and the no-forgetting claim is asserted from the replay mechanism, not yet from a measured before/after retention curve. Read §4–5 as the design; treat its validation as future work.

Around the language model runs a controller ("Pragnosia") that decides, for any input, whether to answer, abstain, learn, seek, or think entirely from the model's own signals, with no keyword rules. Four design commitments distinguish it.

Nothing hardcoded.

Every internal scale is derived from data and re-derived by a `recalibrate()` step as the model grows. Word importance is the self-information `−log p(token)` from the corpus (function words emerge low-weight, content words high, with no stopword list). Honesty, novelty, and memory-match thresholds are set at the equal-error-rate crossover of two of the model's *own* distributions (e.g. agreement on familiar text vs. agreement by chance), not hand-tuned. The training learning rate is found by a Smith range-test rather than seeded, and restored from a sidecar on resume; the growth bar is the data budget (Chinchilla ≈ 20 tokens/param) and VRAM, not a fixed number.

Surprise-modulated continual learning.

When taught a fact, the controller sets its own learning rate from its own surprise prediction error relative to its familiarity boundary learning hard on surprising input and gently on familiar input, and stopping once the fact ceases to surprise it. Teaching interleaves *generative replay* of the training distribution so a new fact does not erase old skills; in practice taught facts persist across sessions and prior abilities are intended to be retained (the replay is designed to bound forgetting; we have not yet measured a before/after retention curve on a benchmark, so we state this as a mechanism, not a result). A persistent in-weights **episodic memory** complements this: it writes surprise-gated key→value traces of the model's own activations, recalls them under an *uncertainty gate* (so it never contaminates confident outputs), decays/forgets over time, and consolidates into the slow weights during a "sleep" step. It is a learned in-weights store, **not** retrieval or RAG there is no external index at recall time.

Algorithm 1 Surprise-modulated continual update (teach)

```
# inputs: fact f, model M, replay corpus R, own familiarity boundary β, base lr
s ← surprise(M, f)   # M's own prediction error on f
η ← base_lr · clamp(s / β, 0.15, 3)   # plasticity = own surprise / own boundary
A ← { (x, argmax M(x)) : x ~ R }   # snapshot M's own predictions = self-anchors
repeat
  L ← CE(M, f) + λ_r·CE(M, x~R) + λ_a·CE(M, A)   # fact + generative replay + self-anchor
  M ← step(M, ∇L, η)
until surprise(M, f) < 0.4·β   # learned enough to recall, not over-memorized
```

Introspection and self-growth.

An introspection layer adds a confidence self-estimate (the model scoring its own certainty), step-by-step deliberation (an internal scratchpad), and idle self-generation. Growth is function-preserving and fires only on *probe-confirmed saturation*: the model spawns a depth-grown and a width-grown copy, trains each briefly, and

commits to growth only if validation loss drops past the validation noise choosing depth vs. width by whichever helps more, capped by the derived data budget and VRAM. Because the diagonal carrier is d-shaped, growth composes with it directly. This rule is confirmed in production: a spin-dominant run grew itself `24M (4 layers)` → `27.3M (5 layers)` on confirmed saturation, then stopped on its own at the data budget the correction to the earlier over-growth to 1.4B.

6. Experiments and Results

Trained instances.

The diagonal-complex carrier trains stably from 21M to 1.4B. Two instances anchor the analysis of §3: the self-grown **1.4B** single-carrier model (val ppl 17.35 after adaptive-lr refinement) is the attention-dominant "wrong build" bigger, but with a causally-negligible carrier; the **284M** spin-dominant model (val ppl 24.54) is the corrected build, 5× smaller, with a provably load-bearing carrier. At these scales the models are coherent at the sentence level, write simple correct code (`def add(a,b):` → `return a + b`), and produce the *form* of step-by-step reasoning; single-step facts and arithmetic are real while multi-hop composition is weak the undertraining-for-size signature, expected before the compute-optimal token budget is reached. The training corpus (`window2_train`) is 330 GB / 176.6B tokens 158.6B of base text plus ~18B of appended chain-of-thought reasoning, targeting that weak axis. The CoT split is assembled from public, openly-licensed reasoning datasets (OpenMathReasoning, OpenThoughts2, OpenMathInstruct, NuminaMath); the base text is Cosmopedia, Wikipedia, and FineWeb-Edu all public, so the corpus is reproducible.

Capability and honesty of the 284M model.

A greedy capability battery on the 284M shows a clear jump from a 27M laptop model (27M in parentheses): arithmetic **3/3** (0/3), knowledge **2/3** (0/3), logic **2/2**, theory-of-mind **2/2** (1/2), code 1/2, in-context binding 1/2; the still-weak axis is multi-step 0/2, counterfactual 0/2, causal 0/2. Sample generations: "The capital of France is → the city of Paris.", "2 + 2 = → 4.", "Once upon a time →, in a small town named Harmonyville, lived two best friends". World knowledge, single-step arithmetic, and theory-of-mind genuinely *emerged* at this scale; multi-step composition is the next axis.

> **An honest negative on honesty.** The controller answers only when its sampled answers *agree* on a content token (real knowledge pins one answer; confabulation varies). Asked in the model's trained chat format this works for some facts, but at 284M the signal does *not* cleanly separate knowledge from confident confabulation: a known fact ("capital of France") and an unknowable ("my secret password", "who wins the 2031 election") score 0.53 / 0.53 / 0.67 overlapping. The model confabulates *consistently*. The gate errs toward "I don't know" rather than guessing; reliable honesty needs either scale (consistency sharpens with model size) or a different confidence signal. We do not claim the chat form is reliably honest.

Zero-shot public benchmarks.

For external comparability we evaluate the 284M model zero-shot on standard public sets. Because the model uses a custom 16K-token vocabulary, we report *word-normalized* perplexity (tokenizer-independent) and log-likelihood multiple-choice accuracy (also tokenizer-independent). The scores are honest and modest above chance on most tasks, at chance on the hardest (ARC-Challenge) the expected profile of a small model undertrained for its size; the evaluation harness (`eval_external.py`) is released.

Table 5. Zero-shot public benchmarks (1,000-example subsamples; `acc_norm` = length-normalized log-likelihood; LAMBADA accuracy is on the first token of the final word).

Benchmark	Metric	Pragnosia 284M	Chance
WikiText-2	word ppl (lower better)	157.9	
LAMBADA	accuracy	19.0%	
HellaSwag	acc_norm	27.9%	25%
ARC-Easy	acc_norm	**34.7%**	25%
ARC-Challenge	acc_norm	24.0%	25%
PIQA	accuracy	**59.5%**	50%

Comparison with open models of similar size.

To place these numbers in context, we ran four open models through the *identical* harness (same example subsamples, same log-likelihood scoring): GPT-2 (124M), Cerebras-GPT-256M, and GPT-2-medium (355M), which bracket our 284M.

Table 6. Zero-shot accuracy (%) vs open models of similar size, one harness, 1,000-example subsamples. Honest summary: comparable to GPT-2 (124M) and Cerebras-256M, clearly behind the larger GPT-2-medium (355M, which it trails on four of five tasks); ARC-Challenge sits at chance for all four models, so that column shows the harness works rather than any parity.

Model	Params	PIQA	ARC-Easy	ARC-Chal.	HellaSwag	LAMBADA
Pragnosia (ours)	284M	59.5	**34.7**	24.0	**27.9**	19.0
Pragnosia (ours), 565M	565M	60.7	34.1	24.9	**29.3**	21.7
GPT-2	124M	63.3	34.1	23.1	27.2	32.2
Cerebras-GPT-256M	256M	62.1	32.7	22.1	26.3	29.5
GPT-2-medium	355M	66.5	37.5	23.3	36.8	42.8

The reading is honest and informative. On **ARC-Easy** our 284M (34.7) matches or beats GPT-2 (34.1) and Cerebras-256M (32.7), trailing only the larger GPT-2-medium; on **HellaSwag** it edges the similar-size models; ARC-Challenge is at chance for all four. But on **PIQA** and especially **LAMBADA** our model is clearly behind — LAMBADA (last-word prediction) rewards fluent general language modeling, which our model, trained on a reasoning-heavy corpus and undertrained for its size, does least well. We include this not to claim a competitive small LM — that is not the contribution — but to benchmark our numbers honestly: the 284M sits in the same range as GPT-2 / Cerebras of similar size, strong on reasoning-style tasks and weak on raw language fluency. A **565M** continuation (val ppl ~22) confirms the trend: it improves on most tasks PIQA 59.5→60.7, ARC-Challenge 24.0→24.9, HellaSwag 27.9→29.3, and LAMBADA 19.0→21.7 (the largest gain, on the very task that most rewards fluency) so the gaps narrow with more training, as expected for an undertrained model.

7. Comparison with Large Language Models

Table 7. Where each paradigm wins. The two are built on opposite bets.

Property	Today's LLMs	Pragnosia
Fluent language / world knowledge	**Strong / vast**	**Decent / limited (small)**
Reasoning depth	**Strong**	**Single-step real, multi-hop weak**
Long-context / inference cost	**$O(T^2)$, KV-cache grows**	**$O(T \cdot d)$, recurrent O(1)/token *(structural; the carry is implemented but untrained currently a liability, §Limitations v)***
Learn after training, no retrain	**No (frozen)**	**Yes (replay + plasticity)**
Forgetting on update	**Catastrophic**	**Low (generative replay)**
Self-paced learning rate	**Fixed schedule**	**Surprise-modulated**
Hand-set behavior		**None (self-derived)**
Knows what it doesn't know	**Partial, bolted on**	**Abstains, but signal scale-limited**
Size / cost	**Data-center**	**Single laptop GPU**

8. Limitations

Every claim above carries a number; the open frontier is equally explicit. (i) **The decisive at-scale experiment is missing, and the headline rests on one seed.** We separate two claims: the *quality* win (spin-dominant beats a matched transformer) is shown at ≈37M, 3 seeds; the *causal-dominance* property (the carrier is load-bearing) is shown at 284M, one seed. These are not the same claim, and the 306× number shows the carrier is *used*, not that the design *wins* at scale. The single highest-value experiment a `carrier="none"` transformer at 284M, same corpus and budget is what would join them; it is not a footnote but the crux, and it is still to run. The asymmetry is also backwards from ideal: the cheap result is replicated (3 seeds) and the expensive headline is not, so a second 284M seed (even at reduced budget) is needed to know whether 306× is stable. (ii) **The models are undertrained for their grown size and we treat this as a claim to be falsified, not a shield.** Single-step reasoning, arithmetic, and fact recall are real; multi-hop composition (relational analogy, multi-step deduction, counterfactual and causal chains) is weak. Our best evidence that tokens are the lever is the 565M: more training raised the most fluency-dependent task most (LAMBADA 19.0→21.7). A direct 2-point check on the multi-hop axis itself (284M vs 565M, greedy battery of analogy / multi-step / counterfactual / causal) nudges `2/10 → 3/10` the gain in counterfactual, while analogy and causal stay at floor: consistent with the account but within the noise of a 10-item probe. To make this falsifiable rather than convenient, the missing measurement is the full *token-vs-capability curve on the multi-hop axis* if more tokens do not move it, the undertraining account is wrong, and we will report it. (iii) **Honesty is not solved.** The consistency-based gate abstains on the clearly-unknowable but cannot separate knowledge from confident confabulation at 284M (§5); it needs scale or a redesigned signal. (iv) Like all small models it can be *confidently wrong* on familiar-but-incorrect facts. (v) **Long context is structural, not yet realized.** The carrier is `O(T·d)` and the windowed-generation path keeps attention positions in-distribution, so the architecture *supports* unbounded context; but our models were trained at a fixed 256-token window with each window starting fresh, so they never learned to *use* a cross-window carried state feeding one to the 565M does not help (next-window ppl 62.6 with the carry vs 58.0 without). We have implemented the cross-window state-threading; realizing the benefit requires *training* with it (a longer effective context), which we have not yet done. None of these is hidden.

9. Conclusion

Pragnosia is a transformer/SSM hybrid in which a diagonal-complex spin recurrence is the core token-mixer, wrapped in a self-calibrating controller for continual learning, a persistent in-weights episodic memory, an introspection layer, and self-governed growth with no hand-set constants. The central, honest finding is about *placement*: a single spin carrier drifts to a causally-negligible side-channel as the model scales (0.028% of the loss at 1.4B), and a param-matched ablation shows the intended spin-dominant design spin as the core mixer is the most parameter-efficient (219 vs 279 ppl). At 284M, the first at-scale training of the intended build, ablating the carrier costs **306×** in perplexity versus 3.4× for attention: the spin carrier is the load-bearing core, the exact inverse of the 1.4B drift. The remaining work is a matched transformer baseline and multi-seed replication at 1B, and training the models to their compute-optimal token budget. We release the full system, every measured number, and every limitation.

---

[1] Vaswani, A. et al. Attention Is All You Need. *NeurIPS*, 2017.

[2] McCloskey, M. & Cohen, N. Catastrophic Interference in Connectionist Networks. *Psychology of Learning and Motivation*, 1989.

[3] Gu, A. & Dao, T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv:2312.00752*, 2023.

[4] Orvieto, A. et al. Resurrecting Recurrent Neural Networks for Long Sequences (LRU). *ICML*, 2023.

[5] Smith, L. N. Cyclical Learning Rates for Training Neural Networks. *WACV*, 2017.

[6] Robins, A. Catastrophic Forgetting, Rehearsal and Pseudorehearsal. *Connection Science*, 1995.

[7] Doya, K. Metalearning and Neuromodulation. *Neural Networks*, 2002.

[8] Gu, A. et al. Efficiently Modeling Long Sequences with Structured State Spaces (S4). *ICLR*, 2022.

[9] Hoffmann, J. et al. Training Compute-Optimal Large Language Models (Chinchilla). *NeurIPS*, 2022.

[10] Peng, B. et al. RWKV: Reinventing RNNs for the Transformer Era. *EMNLP Findings*, 2023.

[11] Sun, Y. et al. Retentive Network: A Successor to Transformer for Large Language Models. *arXiv:2307.08621*, 2023.

[12] Ramsauer, H. et al. Hopfield Networks is All You Need. *ICLR*, 2021.